
INF1350

Sistemas Reativos

Lua + LÖVE

Prof.: Adriano Branco

abranco@inf.puc-rio.br

Slides baseado em aulas anteriores da Prof. Noemi Rodriguez

Lua no mundo real



Criada na PUC-Rio em 1993 → usada no mundo inteiro

Três grandes áreas:

 Jogos

 Ferramentas e aplicações

 Sistemas embarcados e IoT

Uma pequena linguagem que acabou dentro de sistemas muito diferentes.





Lua nos jogos

Lua está presente em milhares de jogos distribuídos pela Steam: 2.400+ jogos com Lua detectado pelo SteamDB
(levantamento automático a partir dos arquivos dos jogos)

Clássicos

- Grim Fandango LucasArts
- Baldur's Gate BioWare
- **World of Warcraft** Blizzard
- Far Cry Crytek

Mais recentes

- **Dota 2** Valve
- **Hades** Supergiant Games
- Factorio Wube Software
- Garry's Mod Facepunch
- Project Zomboid The Indie Stone

Roblox → Luau

Linguagem derivada de Lua usada para programar experiências no Roblox.

Lua é usada para programar comportamento, lógica e extensões de jogos.



Lua dentro de aplicações

Adobe Photoshop	Lightroom	Aplicação e plugins
Wireshark		Dissectors, automação e extensões
Nmap		NSE — Nmap Scripting Engine
Neovim		Plugins e configuração
VLC		Extensões/scripts
MediaWiki / Wikipedia		Scribunto → módulos em Lua
Blackmagic Fusion		Scripting e automação
LuaTeX		Lua como linguagem de extensão do TeX
Solar2D		Engine de jogos baseada em Lua
OBS Studio		Automação, scripts e extensões em Lua

A aplicação é escrita em uma linguagem. Lua entra como linguagem de extensão.

Lua em embarcados e IoT

Lua também aparece em sistemas com recursos limitados

NodeMCU	ESP8266/ESP32 → programação em Lua
OpenWrt / LuCI	Roteadores e equipamentos de rede
Ginga	Middleware da TV Digital brasileira
Advantech Routers	Lua como aplicação/script no roteador
Sierra Wireless	Lua em plataformas de conectividade/M2M
Lua + C/C++	Lua embutida em aplicações e dispositivos

No NodeMCU, Lua deixa de ser apenas uma linguagem de extensão: ela é a linguagem que usamos para programar o dispositivo.

Por que Lua em Sistemas Reativos?

- **Pequena e simples** → foco nos conceitos de programação
- **Funções são valores** → callbacks e eventos
- **Closures** → encapsulamento de estado
- **Corotinas** → concorrência cooperativa
- **Tabelas** → estruturas de dados e estado
- **Extensível e embutível** → integração com C/C++
- **Jogos** → vamos usar LÖVE
- **Embarcados** → vamos usar NodeMCU
- **Criada na PUC-Rio** → uma linguagem desenvolvida aqui e usada no mundo

Lua

<https://www.lua.org/>

Lua

- mais uma linguagem dinâmica
 - alguma similaridade com Python, Perl, e outras
- linguagem de descrição de dados
- ênfase em scripting
 - comunicação inter-linguagens

dinamismo

- tipagem dinâmica
 - verificações em tempo de execução: valores têm tipo
 - tipagem forte: linguagem não aplica operações a tipos incorretos
- interpretação dinâmica de código
 - linguagem capaz de interpretar código dinamicamente no mesmo ambiente de execução do programa

como usar Lua

stand alone

VS

embutida em algum programa

www.lua.org

usando o interpretador

```
$ lua -e "print(2^0.5)"
```

```
$ lua nome-do-arquivo
```

```
$ lua  
> print(2^0.5)
```

```
$ lua  
> dofile("nome-do-arquivo")
```

tipos

<https://www.lua.org/manual/5.4/manual.html#2.1>

- number
- string
- boolean
- nil
- table
- function
- thread → Coroutines
- userdata → C data structures/buffers

tipos

- number
- string
- boolean
- nil

armazena valor

- table
- function
- thread
- userdata

→ Coroutines

→ C data structures/buffers

armazena referência

number

- double (float-point)
- integer

The type *number* represents both integer numbers and real (floating-point) numbers, using two subtypes: *integer* and *float*. Standard Lua uses 64-bit integers and double-precision (64-bit) floats, but you can also compile Lua so that it uses 32-bit integers and/or single-precision (32-bit) floats. The option with 32 bits for both integers and floats is particularly attractive for small machines and embedded systems.

Boolean

- sem exclusividade em testes
- operadores booleanos operam sobre todos os tipos
- nil e false testam como negativos

```
print (false or true) → true
```

```
print (0 or true) → 0
```

```
true
```

```
print (0 or 6) → 0
```

```
print (nil or 10) → 10
```

```
print (x or 1) → 1
```

```
print (false and true) → false
```

```
print (0 and true) →
```

```
print (0 and 6) → 6
```

```
print (nil and 10) → nil
```

```
print (x and 1) → nil
```

soma elementos array

```
function add (a)
    local sum = 0
    for i = 1, #a do
        sum = sum + a[i]
    end
    return sum
end

print(add({10, 20, 30.5, -9.8}))
```

soma elementos array

```
function add (a)
    local sum = 0
    for i = 1, #a do
        sum = sum + a[i]
    end
    return sum
end

print(add({10, 20, 30.5, -9.8}))
```

soma elementos array

```
function add (a)
  local sum = 0
  for i = 1, #a do
    sum = sum + a[i]
  end
  return sum
end

print(add({10, 20, 30.5, -9.8}))
```

soma linhas de arquivo

```
function addfile (filename)
    local sum = 0
    for line in io.lines(filename) do
        sum = sum + tonumber(line)
    end
    return sum
end
```

io

io.read – padrões:

"*a" reads the whole file

"*l" reads the next line (without newline)

"*L" reads the next line (with newline)

"*n" reads a number

num reads a string with up to num characters

Funções em Lua

- funções são valores dinâmicos de primeira classe

```
function (a, b) print (a+b) end
```

*** Uma entidade é de primeira classe quando pode ser usada como um valor comum da linguagem.*

Funções em Lua

- funções são valores dinâmicos de primeira classe

```
(function (a, b) print (a+b) end) (10, 20)
```

```
table.sort(t, function (a,b)  
    return a < b  
end)
```

*** Uma entidade é de primeira classe quando pode ser usada como um valor comum da linguagem.*

Funções e variáveis

- usamos a sintaxe convencional para armazenar funções em variáveis

```
inc = function (a) return a+1 end
```

```
function inc (a)  
    return a+1  
end
```

Múltiplos retornos

- funções em Lua podem retornar múltiplos valores
- atribuição múltipla e ajuste de valores

```
function xxx (a,b)  
    return a*b, a/b  
end  
x1, x2 = xxx(10,20)
```

```
dia, mes, ano = 10, "jan", 2023
```

Escopo

- variáveis locais e globais

```
local a = 5
print(a) --> 5
do
    local a = 6 --create a new local inside the do block
    print(a) --> 6
end
print(a) --> 5
```

Escopo léxico e closures

```
local function f(x)
  local v = x or 0
  return function ()
    local val = v
    v = v + 1
    return val
  end
end
. . .
```

cont1, cont2 = f(), f(7)

```
cont1 = function ()
  local val = v
  v = v + 1
  return val
end
```

$V_0 = 0$

```
cont2 = function ()
  local val = v
  v = v + 1
  return val
end
```

$V_0 = 7$

*** Closure é uma função que mantém acesso às variáveis do ambiente onde foi criada, mesmo depois que esse ambiente deixou de existir.*

Escopo léxico e closures

```
local function f(x)
  local v = x or 0
  return function ()
    local val = v
    v = v + 1
    return val
  end
end
. . .
```

```
. . .
cont1, cont2 = f(), f(7)
print(cont1())    → 0
print(cont1())    → 1
print(cont2())    → 7
print(cont1())    → 2
print(cont1())    → 3
print(cont2())    → 8
```

*** Closure é uma função que mantém acesso às variáveis do ambiente onde foi criada, mesmo depois que esse ambiente deixou de existir.*

Regiões geométricas

- a função abaixo cria regiões circulares:

```
function circle (cx, cy, r)
    return function (x, y)
        return (x - cx)^2 + (y - cy)^2 <= r^2
    end
end
```

Verifica se a distância do ponto (x,y) ao centro (cx,cy) é menor ou igual ao raio r.

Regiões geométricas

- a função abaixo cria regiões circulares:

```
function circle (cx, cy, r)
    return function (x, y)
        return (x - cx)^2 + (y - cy)^2 <= r^2
    end
end
```

```
c1 = circle(5.0, -3.2, 4.5)
c2 = circle(0, 0, 1)
```

```
c1 = function (x, y)
    return (x - 5.0)^2 + (y - (-3.2))^2 <= 4.5^2
end
```

Regiões geométricas

- a função abaixo cria regiões circulares:

```
function circle (cx, cy, r)
  return function (x, y)
    return (x - cx)^2 + (y - cy)^2 <= r^2
  end
end
```

```
c1 = circle(5.0, -3.2, 4.5)
c2 = circle(0, 0, 1)
```

```
if c1(a, b) then
  -- ponto está no disco
```

Combinando regiões

```
function union (ca, cb)
    return function (x, y)
        return ca(x, y) or cb(x, y)
    end
end
```

Define o espaço ocupado pela **união** das 2 figuras.

```
function inter (ca, cb)
    return function (x, y)
        return ca(x, y) and cb(x, y)
    end
end
```

Define o espaço ocupado pela **intercessão** das 2 figuras.

Combinando regiões

```
function union (ca, cb)
  return function (x, y)
    return ca(x, y) or cb(x, y)
  end
end
```

```
u1 = union (c1, c2)
if u1(0.5, 0.5) then
  -- ponto está na união dos discos
  ...
```

Tabelas

- Única estrutura de dados composta da linguagem
- Pode representar arrays, structs/registros, mapas/dicionários, conjuntos, ...
- Arrays associativos: valores são associados a **chaves** de diferentes tipos
- Podem ser **indexadas numericamente**, como arrays tradicionais
- Podem combinar **índices numéricos e chaves nomeadas**
- Podem conter **valores de qualquer tipo**, inclusive funções e outras tabelas
- **Dinâmicas**: elementos podem ser adicionados ou removidos em tempo de execução

Construtores

- criação e inicialização de tabelas

{}

{x = 5, y = 10}

{"Sun", "Mon", "Tue"}

{[exp1] = exp2, [exp3] = exp4}

Todos os prefixos de uma string

```
function prefixes (s, len)
  len = len or 1
  if len <= #s then
    return string.sub(s, 1, len),
           prefixes(s, len + 1)
  end
end
```

```
print(prefixes("alo")) --> a al alo
t = {prefixes("vazavaza")}
```

Estruturas de Dados (1)

table: array -- **inteiros** como índices

```
a = {} -- new array
```

```
a[1]=23
```

```
a[2]=55
```

```
a={23, 55}
```

Estruturas de Dados (2)

table: array -- **elementos** como índices

```
a = {} -- new array
```

```
a["pato"] = 23
```

```
a[200] = 55
```

```
a[x] = true
```

```
if a[x] then
```

```
...
```

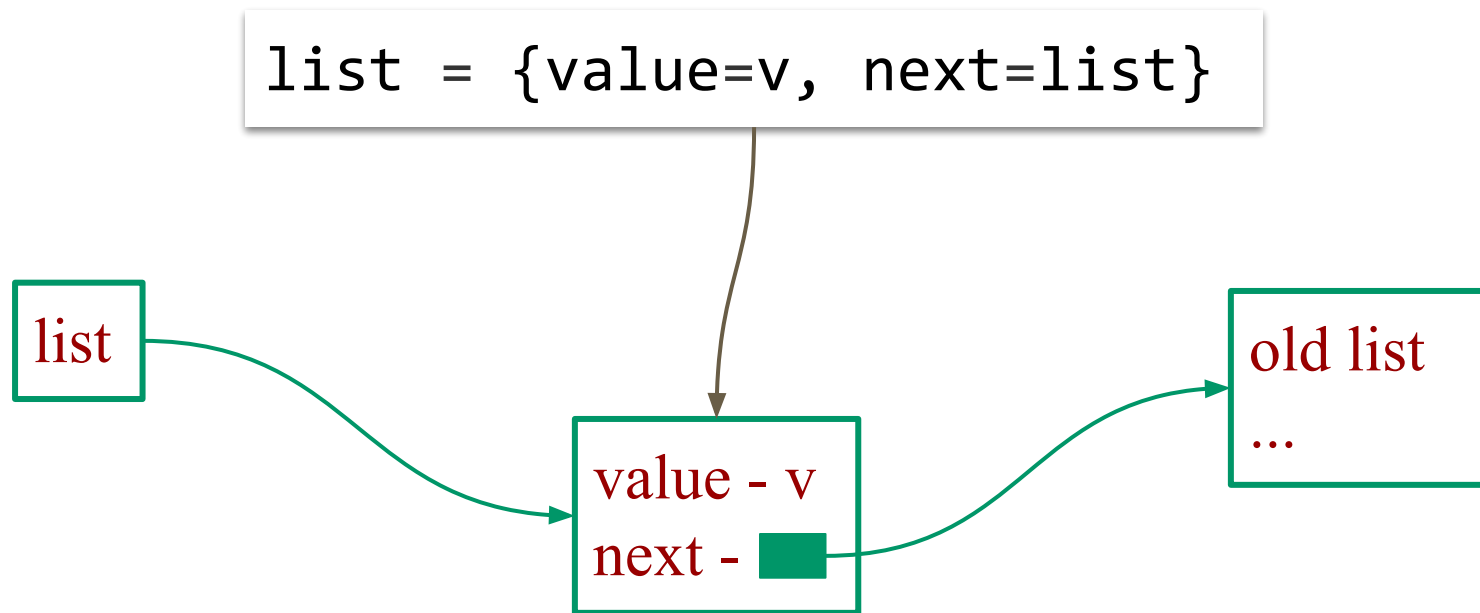
Estruturas de Dados (3)

table: matrix -- **Matriz**

```
mt = {}          -- create the matrix
for i=1,N do
    mt[i] = {}    -- create a new row
    for j=1,M do
        mt[i][j] = 0
    end
end
```

Listas Encadeadas

- Tabelas são **objetos** criados dinamicamente



percurso de tabelas

- arrays indexados por inteiros: **ipairs**
 - *percorre os índices inteiros consecutivos a partir de 1, parando no primeiro índice sem valor.*
- outras tabelas: **pairs**
 - *percorre as chaves da tabela, mas a ordem de iteração não é garantida.*

```
nt = {}  
for k, v in pairs(t) do  
    nt[v] = k  
end
```

Objetos

- funções de 1ª classe + tabelas $\approx$ objetos

```
Rectangle = {w = 100, h = 250}  
function Rectangle.area ()  
    return Rectangle.w * Rectangle.h  
end
```

Rectangle.w	→ 100
Rectangle.h	→ 250
Rectangle.area	→ função

```
function Rectangle.area (self)  
    return self.w * self.h  
end
```

Chamada de métodos

- açúcar sintático para métodos
- cuida do `self`

```
function a:foo (x)  
  ...  
end
```



```
a.foo = function (self,x)  
  ...  
end
```

```
a:foo(x)
```



```
a.foo(a,x)
```

LÖVE

<https://love2d.org/>

löve

- framework para jogos 2D
- Linux, Windows, MacOS, Android, iOS
- love2d.org

löve – estrutura do jogo

- love.load
- love.update
- love.draw

löve: exemplo bobo

```
function love.load()  
  text = "hello!"  
  image = love.graphics.newImage( "alter.jpg" )  
  x = 50 y = 200
```

```
end
```

```
function love.update (dt)  
  x = x + 50*dt  
  if x > 300 then x=50 end
```

```
end
```

```
function love.draw ()  
  love.graphics.draw(image, x, y, 0, 0.1, 0.1)  
  love.graphics.print(text, x, y-50)
```

```
end
```

interação com eventos

- testes no loop como no arduino para eventos contínuos:

```
function love.update (dt)
  x = x + 50*dt
  if love.keyboard.isDown("down") then
    y = y + 10
  end
end
```

- mas também callbacks para eventos discretos
love.keypressed (key)

löve: interação

```
function love.load()
```

```
    x = 50 y = 200
```

```
    w = 200 h = 150
```

```
end
```

```
function naimagem (mx, my, x, y)
```

```
    return (mx>x) and (mx<x+w) and (my>y) and (my<y+h)
```

```
end
```

```
function love.keypressed(key)
```

```
    local mx, my = love.mouse.getPosition()
```

```
    if key == 'b' and naimagem (mx,my, x, y) then
```

```
        y = 200
```

```
    end
```

```
end
```

```
function love.update (dt)
```

```
    ...
```

löve: interação

```
function love.load()  
    x = 50 y = 200 w = 200 h = 150  
end  
  
...  
function love.keypressed(key)  
    ...  
end  
  
function love.update (dt)  
    local mx, my = love.mouse.getPosition()  
    if love.keyboard.isDown("down") and naimagem(mx, my, x, y) then  
        y = y + 10  
    end  
end  
  
function love.draw ()  
    love.graphics.rectangle("line", x, y, w, h)  
end
```

callbacks e estado

```
function love.keypressed(key)
    local mx, my = love.mouse.getPosition()
    if key == 'b' and naimagem (mx,my, x, y) then
        y = 200
    end
end

function love.update (dt)
    local mx, my = love.mouse.getPosition()
    if love.keyboard.isDown("down") and naimagem(mx, my, x, y) then
        y = y + 10
    end
end

function love.draw ()
    love.graphics.rectangle("line", x, y, w, h)
end
```

callbacks e estado

```
function love.keypressed(key)
  local mx, my = love.mouse.getPosition()
  if key == 'b' and naimagem (mx,my, x, y) then
    y = 200
  end
end

function love.update
  local mx, my =
  if love.keyboard
    y = y + 10
  end
end

function love.draw ()
  love.graphics.rectangle("line", x, y, w, h)
end
```

- variáveis globais capturam estado como no Arduino
- mas podemos usar características da linguagem para encapsular esse estado

encapsulando estado

```
function retangulo (x,y,w,h)
  local originalx, originaly, rx, ry, rw, rh = x, y, x, y, w, h
  return {
    draw = function ()
      love.graphics.rectangle("line", rx, ry, rw, rh)
    end,
    keypressed = function (key)
      local mx, my = love.mouse.getPosition()
      ...
    end
  }
end
...
function love.load()
  ret1 = retangulo (50, 200, 200, 150);
end
```

encapsulando estado

```
function love.load()  
    ret1 = retangulo (50, 200, 200, 150);  
End
```

```
function love.keypressed(key)  
    ret1.keypressed(key)  
end
```

exercício 1

1. abrir o ZeroBrane
 - escolher "project"-> "lua interpreter"-> löve
2. executar código em
 - <https://www.inf.puc-rio.br/~abranco/sr-23.2/code/lovelua/retangulo1/main.lua>
 - ("executar" o diretório que contém a main.lua)
3. retirar chamada a `love.keyboard.isDown` e programar a reação à tecla "down" dentro de `keypressed` – entender o que muda!!
4. incluir reação a tecla "right", andando com retângulo para a direita
5. encapsular o retângulo como indicado em slides "encapsulando estado" – Agora `keypressed` e `update` devem chamar chamar função "keypressed" e "update" do retângulo
6. criar um array (tabela) de retângulos

documentação:

- love2d.org/wiki
- www.lua.org
- www.lua.org/pil/contents.html

1ª edição do livro PIL

Tarefa 4 - p.1

1. baixar código em
 - a. <https://www.inf.puc-rio.br/~abranco/sr-23.2/code/lovelua/blip/main.lua>
 - b. esse é um esqueleto de programa (muito feio) em que o jogador é o retângulo de baixo e um inimigo se move na faixa superior – a barra de espaço "lança" um "tiro" (invisível) do centro do jogador para cima. Caso acerte no inimigo, o jogo acaba.
2. modificar o jogo para criar um array (tabela) de blips (em love.load)
 - a. jogo acaba se jogador atingir todos

Tarefa 4 - p.2

3. atribuir a cada blip uma velocidade aleatória (usar `love.math.random`)
 - a. se blip completar X voltas sem ser atingido, deve mudar de desenho e se tornar "imortal"
 - b. se o jogo atingir Y blips imortais, jogador perde
 - c. (acrescentar mais métodos ao blip para implementar essas funcionalidades – procurar deixar o mínimo de estado em variáveis globais)
 4. procure criar novos blips aleatoriamente ao longo do jogo
 5. mostrar tela "ganhou" ou "perdeu"
- Começar na aula. Esse exercício deve ser entregue até a data no ead.